

E-Commerce Order & Inventory Manager

Java Assignment — Storing, Finding, Ordering and Managing Data

1. Assignment Goal

Build a console-based **E-Commerce Order & Inventory Management System**.

The system stores products, customer orders, product categories, orders waiting to be shipped, orders that were already delivered, and customer reviews. Each of these kinds of data behaves differently and has different requirements: some must be fast to look up, some must never contain duplicates, some must remember the order in which things happened, and some must be processed in a strict first-come-first-served sequence.

Your main job is to decide how each kind of data should be stored and how objects should be ordered. The assignment describes the *behavior* the system must have. It is up to you to choose the Java types and techniques that deliver that behavior, and to be able to defend every choice you make.

2. Learning Objectives

By the end of this assignment you should be able to:

- Analyse a business requirement and decide how the data behind it should be stored.
- Provide fast retrieval of a single record when the program only knows its ID.
- Guarantee that a group of values contains no duplicates.
- Process waiting work in strict first-in, first-out order.
- Preserve the exact sequence in which events happened, when that sequence matters.
- Give a class of objects a default ordering, and also support a completely different ordering for another class without touching that class's own code.
- Safely remove elements from a group of stored objects **while** you are looping over it.
- Represent a value that can only ever be one of a small, fixed set of named states.
- Store and manage your own custom objects rather than plain text or numbers.
- Justify every storage and ordering decision you made.

3. Required Classes

Product

Attributes: `int id`, `String name`, `double price`, `String category`, `int stockQuantity`

Required behavior: appropriate constructors, getters and setters, and a `toString()` that prints the product in a readable form.

Ordering requirement: A product has one *default*, built-in way of being ordered: **by price, from cheapest to most expensive**. Any part of the program that asks for products “in order” — without saying which order it wants — must get them sorted by price ascending. This ordering rule belongs to the `Product` class itself; the code that performs the sorting must not have to explain how two products compare.

CartItem

Attributes: `Product product`, `int quantity`

Required methods: `getProduct()`, `getQuantity()`, `calculateSubtotal()`. The subtotal is the product price multiplied by the quantity.

Order

Attributes: `int orderId`, `String customerName`, the items belonging to this order, `double total`, and the current status of the order.

Required methods: `addItem()`, `removeItem()`, `calculateTotal()`, `displayOrder()`, and a method to update the order status.

Storage of the items: The items of an order must be kept in the exact sequence in which the customer added them, and the same product may need to be reached by position while you loop through the order.

Automatic total calculation: The order’s total must not depend on the user remembering to ask for it. Every time `addItem()` or `removeItem()` is called, the method must internally call `calculateTotal()` before returning, so the `total` field is always up to date. `displayOrder()` simply shows the already-updated total — it must never be responsible for calculating it.

Order Status

Every order carries a status. A status can only ever be **one of these four fixed states**:

- Pending
- Shipped
- Delivered

- Cancelled

Rules for the status:

- An order normally starts as *Pending*.
- When it is added to the shipping list, it becomes *Shipped*.
- When it is marked as delivered, it becomes *Delivered*.
- If the order is cancelled, it becomes *Cancelled*.
- No fifth state may ever exist, and it must be **impossible** for a programmer to accidentally set a status such as "delivred", "Shipped " or "done". A typo in a status must be caught by the compiler, not discovered at runtime.
- Free-form text values must **not** be used to represent the status.

Review

Attributes: `int productId, String customerName, String comment`

A simple class with a constructor, getters and `toString()`. All reviews of all products are kept together in one single flat group, in the order they were added. There is no rating field and no nested grouping per product.

4. The Store Class

This class holds all the data of the store and contains the main store operations.

The store must be able to keep track of the following, and **you must decide how each one is stored**:

The store must keep	Behavior it must provide
All products in the shop	Kept in the order they were added; the program frequently walks through all of them one by one and displays them.
A way to reach a single product from its ID	When the user types a product ID, the program must jump straight to that product. It must not be forced to check the products one by one until it finds a match, even if the shop grows to thousands of products.
Every order ever created	Pending, shipped, delivered and cancelled orders alike. Any order must be reachable directly from its ID at any time. Orders are never deleted from this record.
Every distinct product category	Each category name appears exactly once, no matter how many products share it. Adding a category that is already known must silently change nothing. The sequence in which categories come out does not matter.
Orders waiting to be shipped	Orders leave this group in exactly the same sequence they entered it: the order that has waited longest is always the next

The store must keep	Behavior it must provide
	one shipped.
Orders that were delivered	When these are displayed, they must appear in the exact sequence in which they were delivered — first delivered first — and each must still be reachable by its order ID.
All customer reviews	One single flat group, kept in the order the reviews were written.

Permanent order record rule: The record of all orders is the permanent history of the program. An order stays in it after it is shipped, delivered or cancelled. Its status value tells the store what happened to it. Never delete an order from this record.

5. Required Menu

1. Add Product
2. Remove Product
3. Display All Products
4. Search Product by ID
5. Show All Categories
6. Display Products Ordered by Price
7. Create Order
8. Add Item to Order
9. Remove Item from Order
10. Display Order
11. Add Order to the Shipping List
12. Ship Next Order
13. Cancel Order
14. Search Order by ID
15. Add Review to a Product
16. Show All Reviews for a Product
17. Remove Out-of-Stock Products
18. Display Orders Ordered by Total
19. Exit

6. Operation Requirements

Add Product

Read the product's ID, name, price, category and stock quantity. The product must afterwards be visible in the shop's product listing **and** be directly reachable from its ID. Product IDs must be unique — adding a product with an existing ID must be

rejected. The product's category must also be registered among the shop's known categories, without creating a duplicate category.

Remove Product

Remove a product using its ID. If the product does not exist, display an appropriate message.

Consistency rule: Your program will hold the same products in more than one place — one arrangement for listing them and another for reaching them instantly by ID. These two views must never disagree. Whenever a product is deleted, it must disappear from **every** place that holds it, in the same operation; otherwise a search by ID will still return a product that is no longer in the shop. Write a single private helper method (for example `deleteProductEverywhere(int id)`) and call it from *every* operation that deletes a product.

Display All Products

Display all products in the shop, in the order they were added.

Search Product by ID

Ask for an ID and display the matching product if it exists. This lookup must go straight to the product; it must not scan through the whole shop.

Show All Categories

Display every distinct category currently known to the shop. Each category appears exactly once. Any display sequence is acceptable.

Display Products Ordered by Price

Display all products from cheapest to most expensive.

- Use the product's own **default ordering rule** (see the `Product` class). The sorting code here must not describe how to compare two products — that knowledge already lives inside `Product`.
- Sort a **copy**. The shop's own product listing must still be in its original order afterwards.

Create Order

Read an order ID and a customer name. Order IDs must be unique. The new order starts as *Pending* and is added to the permanent order record, where it stays for its whole lifetime.

Add Item to Order

Find the order by its ID, find the requested product by its ID, read the quantity and add the item to the order.

- The product must exist in the shop; otherwise reject the operation.
- Items must **not** be added to an order that is already *Shipped*, *Delivered* or *Cancelled*.
- If the same product is already in the order, **do not add a second entry** — increase the quantity of the existing entry instead.
- After the item is added, the order's total must be recalculated automatically inside `addItem()`. The user must never trigger the calculation manually.

Remove Item from Order

Find the order and remove the requested item from it. Items must not be modified once the order is *Shipped*, *Delivered* or *Cancelled*. After the removal, the total must be recalculated automatically inside `removeItem()`.

Display Order

Display the customer name, order ID, status, every item with its quantity and subtotal, and the final total.

Add Order to the Shipping List

Find the order by its ID and place it at the back of the shipping list, then change its status from *Pending* to *Shipped*.

- The same order must never appear twice in the shipping list.
- An order with no items in it cannot be placed in the shipping list.

Ship Next Order

Take the order that has been waiting **longest** in the shipping list, remove it from that list, change its status to *Delivered* and record it among the delivered orders. It stays in the permanent order record as well.

Before processing, check two things:

1. The shipping list is not empty.
2. The order at the front of the list actually contains at least one item.

An order with no items must **not** be marked as delivered — display an appropriate message (for example “This order has no items and cannot be shipped”) and leave it in the shipping list.

An order must never be shipped or delivered twice.

Cancel Order

Let the store cancel an order that has not been delivered yet.

- Ask the user for an order ID. Find it in the permanent order record; if it does not exist, display an appropriate message and stop.
- If the order is already *Delivered* or already *Cancelled*, reject the cancellation with an appropriate message.
- If the order is *Shipped*, it must also be taken out of the shipping list, since a cancelled order should no longer be waiting to ship.
- If the order is *Pending*, it was never queued, so there is nothing to take out.
- In every valid case, set the order's status to *Cancelled*.
- The order must stay in the permanent order record forever, and it must **never** be recorded among the delivered orders — it was cancelled, not delivered.

Search Order by ID

Find and display any order by its ID, whatever its status is. This lookup must go straight to the order rather than scanning every order in the system.

Add Review to a Product

Read a product ID, a customer name and a comment, then store the review. A review may only be added for a product ID that exists in the shop.

Show All Reviews for a Product

Ask for a product ID, then walk through all stored reviews and display only those written about that product.

Remove Out-of-Stock Products

Delete **every** product whose stock quantity is 0 from the shop, in a single pass.

- You must remove the products **during** the same loop that examines them. Collecting them first and deleting them afterwards is not accepted, and neither is a plain enhanced for loop — attempting the deletion that way will crash your program at runtime, and part of this task is to discover why and to use the mechanism that makes deletion-while-looping legal.
- **Reminder:** deleting a product from the shop's listing does not automatically make it unreachable by ID. After removing it, the same product must also become impossible to find through *Search Product by ID*. Reuse the same helper method described in *Remove Product*.

Display Orders Ordered by Total

Display all orders sorted by their total value.

- This is a **different** ordering rule from the one products use, and it must live **outside** the Order class, as its own separate ordering rule that can be handed to the sorting code.
- Product's own default ordering must not be touched or changed to make this work.
- Sort a **copy** — the permanent order record must not be disturbed.

7. Order Lifecycle

An order moves through these states while always remaining in the permanent order record:

- Create Order → **Pending**
- Add Order to the Shipping List → **Shipped**
- Ship Next Order → **Delivered**
- Cancel Order → **Cancelled**

The permanent record contains the order in every state. The shipping list holds only orders still waiting to ship — an order leaves it either by being shipped or by being cancelled. The delivered-orders record holds delivered orders in delivery sequence.

8. Example Data

Products

- 1 — Wireless Mouse — 250 — Electronics — Stock: 15
- 2 — Notebook — 30 — Stationery — Stock: 0
- 3 — Desk Lamp — 180 — Home — Stock: 8
- 4 — USB Cable — 60 — Electronics — Stock: 20

Example Order #501

- Customer: Sara
- Status: Pending
- Wireless Mouse × 1 = 250
- USB Cable × 2 = 120
- **Total = 370**

9. Important Rules

- Do **not** use plain arrays to hold the store's data.
- Product IDs and order IDs must be unique.
- Every order ever created must remain findable for the whole run of the program, including delivered and cancelled ones.

- An order cannot be shipped when nothing is waiting to be shipped.
- An order cannot be shipped if it contains no items, even if it is next in line.
- An order must never be shipped or delivered twice.
- An item cannot be added to an order if the product does not exist in the shop.
- Adding the same product twice to one order must increase the quantity, not create a duplicate entry.
- Delivered or cancelled orders must not be modified.
- An order cannot be cancelled twice, and a delivered order cannot be cancelled.
- The order total must always be recalculated automatically whenever items are added or removed.
- Out-of-stock products must be deleted during the loop that examines them, not afterwards.
- An order's status must be impossible to set to an invalid value; a wrong status must be a compile-time error.
- Looking up a product or an order by ID must not require scanning through everything.
- The program must survive invalid IDs and empty data without crashing.
- Keep the program console-based; no database and no graphical interface.